

Subscription Cancellation Guarantee

Team Playbook · 13 Screens · 5 Days

TEAM

pranjal · bhavani · yashaswini · vishwas

DEPLOYMENT

Railway · Demo-first · Live-ready

■■ OVERVIEW ■■

What This App Does

A web app that stops users losing money to hidden subscription renewals. The landing page teaches what the app does before asking for login — then guides the user through a complete demo with realistic sample data.

Core User Journey

1	Land on page	Understand the problem and solution — no login required
2	Click Try Demo	See all subscriptions instantly with sample data
3	Click Cancel	See protection options and cancellation methods
4	Get Alert	When a charge happens, generate dispute proof instantly

Design Principles

- Modern, premium, polished — high-value fintech vibes, not a student project.
- Every screen is self-explanatory. No tooltips or help text needed.
- Responsive: works beautifully on phone, tablet, and desktop.

■■ SCREENS ■■

All 13 Screens & Ownership

#	Screen Name	Owner
1	Landing Hero	bhavani
2	Problem Section	yashaswini
3	How It Works Roadmap	bhavani
4	Try Demo Button Section	bhavani
5	Auth / Onboarding	pranjal
6	Dashboard	pranjal
7	Subscriptions List	yashaswini
8	Subscription Detail	pranjal

9	Renewal Calendar	pranjal
10	Cancellation Center	vishwas
11	Protection Controls	vishwas
12	Dispute Studio + Settings	yashaswini
13	Alerts and Incident Feed	vishwas



TIMELINE

5-Day Sprint to Devpost Submission

DAY 1		Effort: 70/100
pranjal	Backend skeleton, database schema, API contracts + QA: API contract review	
bhavani	Design system, component library, color palette + QA: visual consistency checklist	
yashaswini	Product narrative, problem copy, roadmap wireframe + QA: copy and UX review	
vishwas	Railway setup, Docker, CI skeleton, health checks and monitoring	
Deliverable: Runnable app shell + mockups of all 13 screens		
DAY 2		Effort: 95/100
pranjal	Auth API, Dashboard API, user state management + QA: API endpoint testing	
bhavani	Landing Hero (1), How It Works Roadmap (3), Try Demo section (4) + QA: responsive checks	
yashaswini	Problem Section (2), Subscriptions List API contract, demo data schema + QA: copy review	
vishwas	Cancellation Center skeleton, E2E test framework setup, deploy pipeline	
Deliverable: Screens 1–4 complete and demoable. APIs returning stable data		

DAY 3		Effort: 100/100
pranjal	Subscription Detail (8), Renewal Calendar (9), completion logic + QA: backend flow testing	
bhavani	Visual polish on screens 1–4, responsive design pass + QA: cross-device visual testing	
yashaswini	Subscriptions List (7), Dispute Studio (12) mockup, copy approved + QA: UX flow review	
vishwas	Protection Controls (11), Alerts Feed (13), DevOps integration	
Deliverable: Full E2E flow works: landing → demo → view subs → cancel → alert → dispute		

DAY 4		Effort: 85/100
pranjal	Auth edge cases, API stabilization, backend error handling + QA: API regression pass	
bhavani	Mobile responsiveness full pass, visual consistency audit + QA: final visual sign-off	
yashaswini	Dispute Studio full build, DevOps integration (50% with vishwas) + QA: copy final check	
vishwas	Protection Controls full build, Alerts Feed polish, DevOps integration (50% with yashaswini)	
Deliverable: All 13 screens complete and polished. Release candidate ready		

DAY 5		Effort: 60/100
pranjal	Final GitHub README, tech architecture docs, live link verification	
bhavani	Final screenshot package, design consistency final check	
yashaswini	Devpost write-up complete. Submission form filled	
vishwas	Deploy to production, uptime monitoring, backup and rollback ready	
Deliverable: Complete Devpost submission with all required fields. Live demo accessible		

PRANJAL'S ROLE

pranjal — Backend + Data + 4 UI Screens

pranjal

Backend + Data + 4 UI Screens

4 screens

First 2 Hours on Day 1

1. Finalize database schema: users, subscriptions, renewals, cancellations, blocks, disputes
2. Lock API contracts — define all endpoints and their return shapes
3. Create GitHub board with issues assigned by day and owner

What You're Building

- Build backend API (Node.js + Fastify) that serves all app data
- Implement subscription tracking, renewal prediction, and cancellation lifecycle
- Own user authentication and session management
- Build 4 UI screens: Auth, Dashboard, Subscription Detail, Renewal Calendar
- Ensure demo mode works without any external credentials
- Own API QA: test all endpoints with Postman, validate demo data, catch any 500 errors

Definition of Done

- ✓ Auth flow is secure, fast, and clean UX
- ✓ Dashboard API returns monthly spend, next renewal date, and risk level
- ✓ Subscription Detail shows full info, history, and action buttons
- ✓ Renewal Calendar displays all charge dates color-coded by risk
- ✓ All APIs respond under 500ms, no errors in demo data
- ✓ CI passes and Railway deploy succeeds automatically
- ✓ QA sign-off: all API endpoints return correct data, full demo flow error-free

Tools & Stack

Backend	Node.js 20+ · Fastify · TypeScript · Prisma · PostgreSQL · Redis + BullMQ
Frontend	Next.js 14 · React 18 · TypeScript · Tailwind CSS
DevOps	Railway · Docker · GitHub Actions
APIs (post Day 4)	Gmail API · SendGrid

■■ BHAVANI'S ROLE ■■

bhavani — Frontend + Visual Design + 3 UI Screens

bhavani

Frontend + Visual Design + 3 UI Screens

3 screens

First 2 Hours on Day 1

1. Create Figma design system: colors, typography, spacing, buttons, cards, badges
2. Define 3–5 reusable components: Hero, Card, Timeline, CTA, Status Badge
3. Choose responsive breakpoints: 375px mobile, 768px tablet, 1024px+ desktop

What You're Building

- Design and build Landing Hero (1), How It Works Roadmap (3), and Try Demo section (4)
- Create the complete visual design system for the whole product
- Ensure screens 1–4 explain the problem and solution before any signup prompt
- Build a reusable component library for the whole team
- Own visual QA for screens 1–4: cross-device testing, pixel-perfect consistency, final polish pass

Definition of Done

- ✓ Landing Hero: headline is clear in 5 seconds — Detect. Cancel. Block. Prove.
- ✓ How It Works Roadmap: 5 steps with icons, self-explanatory at a glance
- ✓ Try Demo section: prominent and obvious on both mobile and desktop
- ✓ Design system documented in Figma: colors, typography, spacing, button states
- ✓ All reusable components are responsive and production-ready
- ✓ All screens 1–4 visually consistent across phone, tablet, and desktop
- ✓ QA sign-off: screens 1–4 pass visual review on all 3 breakpoints

Tools & Stack

Frontend	Next.js 14 · React 18 · TypeScript · Tailwind CSS
Design	Figma · Lucide Icons
Color Palette	Deep Navy · Emerald · Amber · Slate
APIs	None (pure UI)

■■ YASHASWINI'S ROLE ■■

yashaswini — Product + Narrative + 3 Screens + DevOps 50%

yashaswini

Product + Narrative + 3 Screens + DevOps 50%

3
screens

First 2 Hours on Day 1

- 1. Write the problem statement in 1–2 relatable paragraphs
- 2. Draft the how-it-solves narrative: Detect → Predict → Cancel → Block → Prove
- 3. Create copy guidelines: grade-6 reading level, no jargon

What You're Building

- Write all product narrative and user messaging for the entire app
- Build Problem Section (2): pain → cause → solution narrative
- Build Subscriptions List (7): card layout, filter/sort, status badges
- Build Dispute Studio + Settings (12): evidence, timeline, export, privacy controls
- Own 50% of DevOps: demo data, deterministic backend state, CI/CD pipeline
- Write the complete Devpost description covering all required sections
- Own UX QA for screens 2, 7, 12: verify flows, copy accuracy, and data realism

Definition of Done

- ✓ Problem Section: clearly explains money loss, dark patterns, and how the app helps
- ✓ Subscriptions List: clean cards, filter/sort by status/price/date, professional UX
- ✓ Dispute Studio: evidence timeline, export as PDF, email template, privacy controls
- ✓ All copy at grade-6 reading level, consistent tone throughout
- ✓ Demo data: 10 realistic subscriptions, 3 upcoming renewals, 1 charge incident
- ✓ Devpost write-up: all sections complete — problem, solution, impact, architecture, team
- ✓ QA sign-off: screens 2, 7, 12 pass UX review — no broken flows, no placeholder text

Tools & Stack

Frontend	Next.js 14 · React 18 · TypeScript · Tailwind CSS
Writing	GitHub (docs) · Grammarly
DevOps (50%)	Railway · Docker · Seed scripts · GitHub Actions
APIs	None required for demo

■■ VISHWAS'S ROLE ■■

vishwas — Operational UI + DevOps 50% + 3 UI Screens

vishwas

Operational UI + DevOps 50% + 3 UI Screens

3
screens

First 2 Hours on Day 1

1. Set up Railway project, Docker containers, and CI/CD pipeline skeleton
2. Configure health checks, uptime monitoring, and rollback procedures
3. Create deployment runbook so the team can ship confidently every day

What You're Building

- Build Cancellation Center (10): step-by-step flow with progress indicator
- Build Protection Controls (11): per-subscription Auto-Block toggle with clear status
- Build Alerts Feed (13): severity-coded timeline of notifications
- Own 50% of DevOps: deploy stability, health checks, rollback plan
- Maintain Railway deployment stability throughout Days 3–5

Definition of Done

- ✓ Cancellation Center: shows merchant, method, progress step, and next action clearly
- ✓ Protection Controls: toggle obvious, status shows Active/Inactive, next renewal visible
- ✓ Alerts Feed: severity color-coded, timestamp visible, clear action guidance
- ✓ Railway deploy stable during Days 3–5, uptime 99.9%+, rollback ready
- ✓ Backup environment deployable within 5 minutes if needed

Tools & Stack

Frontend	Next.js 14 · React 18 · TypeScript · Tailwind CSS
DevOps (50%)	Railway CLI · Docker · GitHub Actions · Sentry (optional)
APIs	None

■■ SUBMISSION ■■

Devpost Submission Checklist — Day 5

- ✓ All 13 screens implemented and visually polished
- ✓ App landing page teaches problem + solution before user signs in
- ✓ Try Demo button works — loads sample data instantly
- ✓ Complete user journey works end-to-end without errors
- ✓ Live Railway deployment is stable and accessible
- ✓ GitHub repo is public with a clear setup README
- ✓ Devpost submission has all required fields filled
- ✓ Team has commit history proving originality

■■ DAILY RHYTHM ■■

Standup Questions — Ask These Every Day

All

- ? Are all screen mockups approved?
- ? Is demo mode deterministic — same data every run?
- ? Is every blocker logged?
- ? Did you QA your own screens today? Any broken flows to report?

pranjal

- ? Do all APIs have responses? Any 500 errors in logs?
- ? Can demo mode run without external credentials?
- ? Have all your API endpoints been tested in Postman?

bhavani

- ? Are all your screens responsive across mobile, tablet, desktop?
- ? Do landing screens 1–4 teach users what the app is?
- ? Did screens 1–4 pass your visual QA checklist?

yashaswini

- ? Is all copy under 8th-grade reading level?
- ? Are demo data subscriptions realistic and believable?
- ? Did screens 2, 7, and 12 pass your UX flow review?

vishwas

- ? Is Railway deployment stable with zero downtime?

- ? Are health checks green? Is rollback ready if needed?
- ? Did screens 10, 11, and 13 pass your own button-click review?

Keep scope locked. Keep design clean. Keep demo stable. Finish strong.